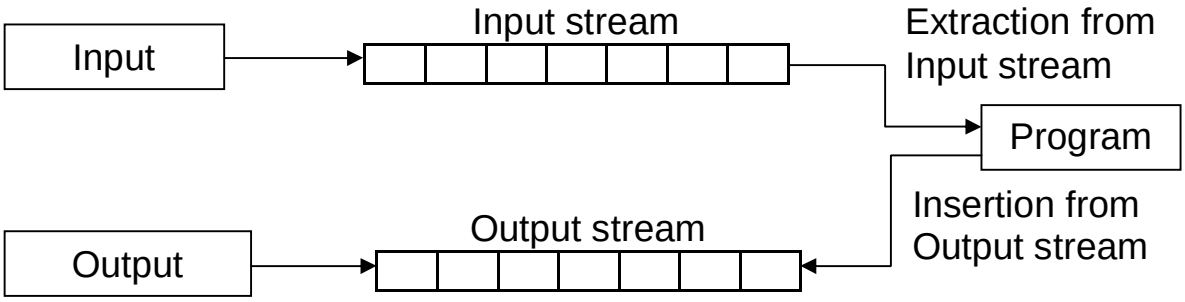
CORE–3: PROGRAMMING USING C++ (UNIT – 4)

Managing Console I/O Operations: Console is the set of devices through which a user communicates with a system using interactive set of commands. GUI is not considered as console in this respect. The primary input device in most of the computer systems is a keyboard and the primary output is the monitor. The pair forms the user's console in most of the cases. There are mainly two types of consol I/O operations form:

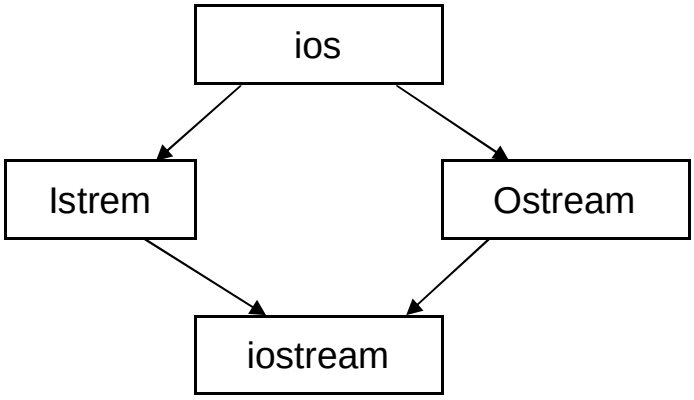
1. Unformatted consol input output
2. Formatted consol input output

C++ Streams: A stream is a sequence of bytes. It acts either as a source from which the input data can be obtained or as a destination to which the output data can be sent. The source stream that provides data to the program is called the **input stream** and the destination stream that receives output from the program is called the **output stream**.



C++ Stream Classes: *Stream classes* in C++ are used to input and output operations on files and I/O devices. These classes have specific features and to handle input and output of the program. The **iostream.h** library holds all the stream classes in the C++ programming language. The following figure shows the stream hierarchy.

Stream classes for console I/O operations:



ios Class − This class is the base class for all stream classes. The streams can be input or output streams. This class defines members that are independent of how the templates of the class are defined.

istream Class − The istream class handles the input stream in c++ programming language. These input stream objects are used to read and interpret the input as a sequence of characters. The **cin** handles the input.

ostream Class − The ostream class handles the output stream in c++ programming language. These output stream objects are used to write data as a sequence of characters on the screen. **cout and puts** handle the out streams in c++ programming language.

iostream Class – It inherits the properties of **ios, istream** and **ostream** through multiple inheritance and thus contains all the input and output functions.

Unformatted I/O Operations: These input / output operations are in unformatted mode. The following are operations of unformatted consol input / output operations:
A) void get() B) void put()

A) void get() : It is a method of cin object used to input a single character from keyboard. But its main property is that it allows wide spaces and newline character.
Syntax:
char c=cin.get();

Example:
```
#include<iostream>
using namespace std;
int main() { char c=cin.get();
        cout<<c<<endl;   return 0;
}
```
Output:
```
I
I
```

B) void put(): It is a method of cout object and it is used to print the specified character on the screen or monitor.
Syntax:
cout.put(variable / character);

Example:
```
#include<iostream>
using namespace std;
int main(){ char c=cin.get();
cout.put(c); //Here it prints the value of variable c;
        cout.put('c'); //Here it prints the character 'c';
        return 0; }
```
Output:
```
I
Ic
```

C) getline(char *buffer,int size): This is a method of cin object and it is used to input a string with multiple spaces.
Syntax: **char x[30];**
cin.getline(x,30);

Example:
```
#include<iostream>
using namespace std;
int main() {
        cout<<"Enter name :";
        char c[10];
        cin.getline(c,10); //It takes 10 charcters as input;
        cout<<c<<endl;      return 0; }
```
Output:
```
Enter name :Divyanshu
Divyanshu
```

D) write(char * buffer, int n):
It is a method of cout object. This method is used to read n character from buffer variable.

Syntax: cout.write(s,n);

where ‘s’ is the string and ‘n’ is the number of character to write.

Example:

```
#include<iostream>
using namespace std;
int main() {
        cout<<"Enter name : ";      char c[10];
        cin.getline(c,10); //It takes 10 charcters as input;
        cout.write(c,9); //It reads only 9 character from buffer c;
        return 0; }
```

Output:

```
Enter name : Divyanshux
Divyanshu
```

E) cin: It is the method to take input any variable / character / string.

Syntax: **cin>>variable / character / String;**

Example:

```
#include<iostream>
using namespace std;
int main(){
   int num;    char ch;    string str;
   cout<<"Enter Number"<<endl;       cin>>num; //Inputs a variable;
   cout<<"Enter Character"<<endl;    cin>>ch; //Inputs a character;
   cout<<"Enter String"<<endl;       cin>>str; //Inputs a string;
   return 0;  }
```

Output:

```
Enter Number
07
Enter Character
h
Enter String
Deepak
```

F) cout: This method is used to print variable / string / character.

Syntax: **cout<< variable / charcter / string;**

Example:

```
#include<iostream>
using namespace std;
int main() {
   int num=100;    char ch='X';    string str="Deepak";
   cout<<"Number is "<<num<<endl; //Prints value of variable;
   cout<<"Character is "<<ch<<endl; //Prints character;
   cout<<"String is "<<str<<endl; //Prints string;
   return 0;  }
```

Output:

```
Number is 100
Character is X
String is Deepak
```

Formatted Console I/O Operations: Formatted console input/output functions are used for performing input/output operations at console and the resulting data is formatted and transformed. They include the following.

1. ios stream class member function and flags
2. Standard manipulator
3. User defined output functions

ios format functions

Functions	Description
width(int width)	Using this function, we can specify the width of a value to be displayed in the output at the console.
precision(int num_of_digts)	Using this function, we can specify the number of digits(num_of_digits) to the right of decimal, to be printed in the output.

In C++, we can read the input entered by a user at console using an object cin of istream class and we can write the output at console using an

object cout of ostream class. Through the cin and cout objects, we can access the formatted I/O functions.

Managing Output with Manipulators: The header file **iomanip** provides a set of functions called **manipulators** which can be used to manipulate the output formats. They provide the same features as that of the **ios** member functions and flags. Some manipulators are more convenient to use than their counterparts in the **class ios**. For example, two or more manipulators can be used as a chin in one statement.

Manipulators and their meanings

Manipulators	Meaning	Equivalent ios functions
setw(int w)	Set the field width to w	width(int width)
setprecision(int d)	Set the floating point precision to d	precision(int num_of_digts)
endl	Insert new line and flush stream.	"\n"

Designing Our Own Manipulators: We can design our own manipulators for certain special purpose. The general form for creating a manipulator without any arguments is as follows.

```
ostream& manipulator (ostream& output) {
..... (code)
return output;
}
```

Here, the output is the name of the manipulator under creation.

Example - The following function defines a manipulator called **unit** that display "inches".

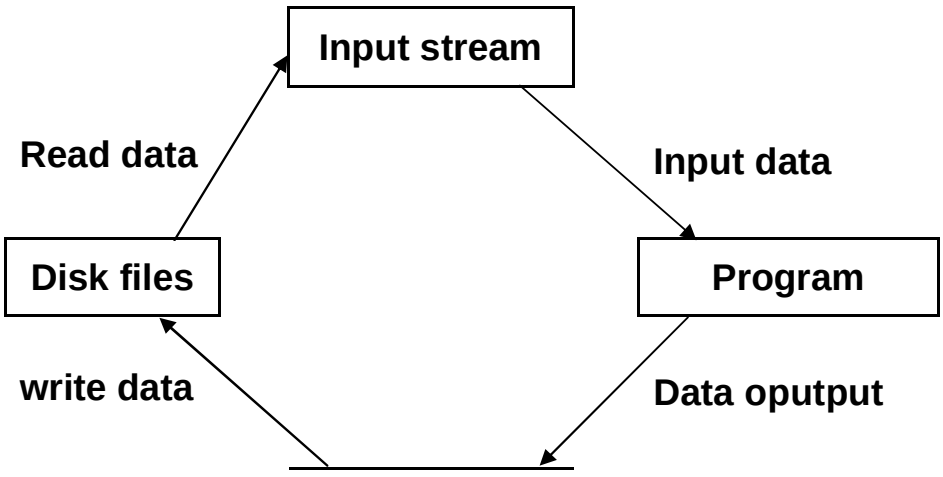
```
#include<iostream>
#include<iomanip>
using namespace std;
ostream& manipulator (ostream& output) {
output << " inches";
return output;
}
Int main() {
   Cout << 36 << unit;  }
```

Output: 36 inches

Files: Working with files generally requires the following kinds of data communication methodologies:

- Data transfer between console units
- Data transfer between the program and the disk file

This figure shows how a program read/write from/to a file.

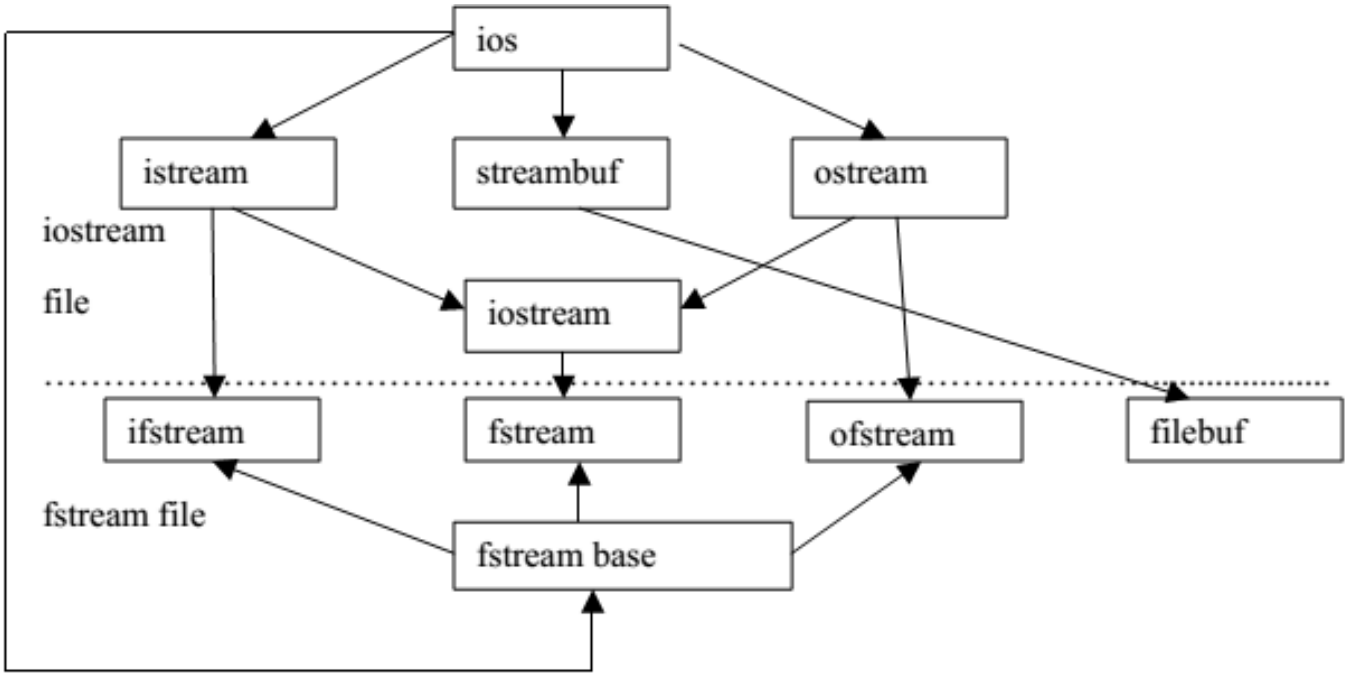


Output stream

File input and output stream

Classes for File Stream Operations: The I/O system of C++ contains a set of classes that define the file handling methods. These include **ifstream, ofstream** and **fstream**. These classes are derived from **fstreambase** and from the corresponding **iostream** class as shown in the following figure.

Stream classes for file stream operation



Stream classes for file operations

Details of file stream classes

Class	Contents
filebuf	Its purpose is to set the file buffers to read and write. Contains **Openprot** constant used in the **open()** of file stream classes.Also contain **close()** and **open()** as members.
fstreambase	Provides operations common to file streams.Serves as a base for **fstream,ifstream** and **ofstream** class.Contains **open()** and **close()** functions.
ifstream	Provides input operations.Contains **open()** with default input mode.Inherits the functions **get(),getline(),read(),seekg(),tellg()** functions from istream.
ofstream	Provides output operations.Contains **open()** with default output mode.Inherits **put(),seekp(),tellp()** and **write()** functions from **ostream**.
fstream	Provides support for simultaneous input and output operations.Contains open with default input mode.Inherits all the functions from **istream** and **ostream** classes through **iostream**.

Opening and Closing a File: To open a file, we must first obtain a stream. There are the following three types of streams:

- input
- output
- input/output

Creating an Input Stream: To create an input stream, we must declare the stream to be of class ifstream. Here is an example: **ifstream fin;**

Creating an Output Stream: To create an output stream, we must declare it as class ofstream. Here is an example: **ofstream fout;**

Create both Input/Output Streams: Streams that will be performing both input and output operations must be declared as class fstream. Here is an example:

fstream fio;

Once a stream has been created, next step is to associate a file with it. And thereafter the file is available (opened) for processing. Opening of files can be achieved in the following two ways :

1. Using the constructor function of the stream class.
2. Using the function open().

The first method is preferred when a single file is used with a stream. However, for managing multiple files with the same stream, the second method is preferred.

Opening File Using Constructors: We know that a constructor of class initializes an object of its class when it (the object) is being created. Same way, the constructors of stream classes (ifstream, ofstream, or fstream) are used to initialize file stream objects with the filenames passed to them. For example, to open a file

named **myfile** as an input file (i.e., data will be fed from it and no other operation like writing or modifying would take place on the file), we shall create a file stream object of input type i.e., ifstream type.
Here is an example: **ifstream fin("myfile", ios::in) ;**

After creating the ifstream object fin, the file myfile is opened and attached to the input stream, fin. Now, both the data being read from myfile has been channelised through the input stream object. Now to read from this file, this stream object will be used using the extraction operator (">>").

Here is an example:
char ch;
fin >> ch ; // read a character from the file and store it in the variable ch.
float amt ;
fin >> amt ; // read a floating-point number form the file and store it in the variable amt.

Similarly, when we want a program to write a file i.e., to open an output file (on which no operation can take place except writing only). This will be accomplish by

1. creating ofstream object to manage the output stream
2. associating that object with a particular file

Here is an example: **ofstream fout("secret", ios::out) ;** //create ofstream object named as fout
This would create an output stream, object named as fout and attach the file secret with it.
Now, to write something to it, you can use << (put to operator) in familiar way. Here is an example,

int code = 2193 ;
fout<<code<<"xyz"; /*will write value of code and "xyz" to fout's associated file named "secret" here. */

Opening Files Using Open() Function: There may be situations requiring a program to open more than one file. The strategy for opening multiple files depends upon how they will be used. If the situation requires simultaneous processing of two files, then we need to create a separate stream for each file. However, if the situation demands sequential processing of files (i.e., processing them one by one), then we can open a single stream and associate it with each file in turn. To use this approach, we declare a stream object without initializing it, and then use a second statement to associate the stream with a file. For example,

```
ifstream fin;                    // create an input stream
fin.open("Master.dat", ios::in);        // associate fin stream with file Master.dat
.....                                   // process Master.dat
fin.close();                     // terminate association with Master.dat
fin.open("Tran.dat", ios::in);          // associate fin stream with file Tran.dat
.....                                   // process Tran.dat
fin.close();                     // terminate association
```

The above code lets us handle reading two files in succession. But the first file is closed before opening the second one. This is necessary because a stream can be connected to only one file at a time.

File Modes: The file mode describes how a file is to be used : to read from it, to write to it, to append it, and so on. When we associate a stream with a file, either by initializing a file stream object with a file name or by using the open() method, we can provide a second argument specifying the file mode, as mentioned below :

stream_object.open("filename", filemode) ;

The second argument of open(), the filemode, is of type int, and we can choose one from several constants defined in the ios class. The following table lists the **filemodes** available in C++ with their meaning:

List of File Modes in C++

Constant	Meaning	Stream Type
ios :: in	It opens file for reading, i.e., in input mode.	ifstream
ios :: out	It opens file for writing, i.e., in output mode. This also opens the file in ios :: trunc mode, by default. This means an existing file is truncated when opened, i.e., its previous contents are discarded.	ofstream
ios :: ate	This seeks to end-of-file upon opening of the file. I/O operations can still occur anywhere within the file.	ofstream ifstream
ios :: app	This causes all output to that file to be appended to the end. This value can be used only with files capable of output.	ofstream
ios :: trunc	This value causes the contents of a pre-existing file by the same name to be destroyed and truncates the file to zero length.	ofstream
ios :: nocreate	This cause the open() function to fail if the file does not already exist. It will not create a new file with that name.	ofstream
ios :: noreplace	This causes the open() function to fail if the file already exists. This is used when you want to create a new file and at the same time.	ofstream
ios :: binary	This causes a file to be opened in binary mode. By default, files are opened in text mode. When a file is opened in text mode, various character translations may take place, such as the conversion of carriage-return into newlines. However, no such character translations occur in file opened in binary mode.	ofstream ifstream

If the **ifstream** and **ofstream** constructors have the default arguments ios :: in and ios :: out respectively for the second argument(mode). So, only filename can be supplied while creating objects. But the **fstream** class does not provide a mode by default and, therefore, one must specify the mode explicitly when using an object of **fstream** class.
We can combine two or more **filemode** constants using the C++ bitwise OR operator (symbol |). For example, the following statement

ofstream fout;
fout.open("Master", ios :: app | ios :: nocreate);

will open a file in the append mode if the file exists and will abandon the file opening operation if the file does not exist.

To open a binary file, we need to specify **ios :: binary** along with the file mode, e.g.,

fout.open("Master", ios :: app | ios :: binary); or
fout.open("Main", ios :: out | ios :: nocreate | ios :: binary);

Closing a File in C++: The connections with a file are closed automatically when the input and the output stream objects expires i.e., when they go out of scope. (For example, a global object expires when the program terminates). Also, we can close a connection with a file explicitly by disconnecting it with the stream it is associated with by using the close() method. The general form of close() function is as follows:

stream_object.close();

For example, if a file Master is connected with an ofstream object fout, its connections with the stream fout can be terminated by the statement, **fout.close() ;**
Closing such a connection does not eliminate the stream; it just disconnects it from the file. The stream still remains there. For example, after the above statements, the streams fin and fout still exist along with the buffers they manage. We can reconnect the stream to the same file or to another file, if required. Closing a file flushes the buffer which means the data remaining in the buffer (input or output stream) is moved out of it in the direction it is ought to be. For example, when an input file's connection is closed, the data is moved from the input buffer to the program and when an output file's connection is closed, the data is moved from the output buffer to the disk file.

Opening and Closing a File Example:

```
#include<iostream>
#include<conio>
#include<string>
#include<stdio>
#include<fstream>
#include<stdlib>
using namespace std;
void main() {
   ofstream fout;
   ifstream fin;
   char fname[20], rec[80], ch;
   clrscr();
   cout<<"Enter file name: ";
   cin.get(fname, 20);
fin.open(fname, ios::in);
if(!fin) {
   cout<<"Error in opening the file "<<fname;
   cout<<"\nPress any key to exit...";
   getch();
   exit(2);
}
cin.get(ch);
fin.get(rec, 80);
cout<<"\nThe file contains:\n";
cout<<rec;
cout<<"\n\nPress any key to exit...\n";
fin.close();
getch();
```

```
fout.open(fname, ios::out);
if(!fout) {
   cout<<"Error in opening the file "<<fname;
   getch();
   exit(1);
}
cin.get(ch);
cout<<"\nEnter a line to store in the file:\n";
cin.get(rec, 80);
fout<<rec<<"\n";
cout<<"\nThe line stored successfully..!!";
cout<<"\nPress any key to see...\n";
getch();
fout.close();
}
```

Output:

```
Enter file name: myfile.txt
Enter a line to store in the file:
How to open, close a file in C++. This is the line.
The line stored successfully..!!
Press any key to see...
The file contains:
How to open, close a file in C++. This is the line.
Press any key to exit...
```

Detecting end-of-file: While reading data from a file, it is necessary to detect the end of file. This can be done using the ***eof()*** function of ***ios*** **class**. It returns 0 when there is data to be read and a non-zero value if there is no data. Following program demonstrates writing and reading multiple rows of data into a file.

```
#include <iostream>
#include<fstream>
using namespace std;
int main() {
   string name;
   int salary;
   ofstream opfile("data.txt");
   for(int i=1; i<=3; i++) {
      cout<<"Enter name and salary:";
      cin>>name>>salary;
      opfile<<name<<"\t"<<salary;
   }
   opfile.close();
   cout<<"Data stored into file."<<endl;
   cout<<"Reading data from file..."<<endl;
   ifstream ipfile("data.txt");
   while(!ipfile.eof()) {   //Checking for end of file
      ipfile>>name>>salary;
      cout<<"\nName: "<<name<<" Salary: "<<salary;
   }
   ipfile.close();
   return 0;
}
```

```
Output for the program is as follows:
Enter name and salary:suresh 7000
Enter name and salary:ramesh 8000
Enter name and salary:mahesh 9000
Data stored into file.
Reading data from file...
Name: suresh Salary: 7000
Name: ramesh Salary: 8000
Name: mahesh Salary: 9000
```

File Pointers and their Manipulations: Every file maintains two pointers called get_pointer (in input mode file) and put_pointer (in output mode file) which tells the current position in the file where reading or writing will takes place. (A file pointer in this context is not like a C++ pointer but it works like a book-mark in a book.). These pointers help attain random access in file. That means moving directly to any location in the file instead of moving through it sequentially. There may be situations where random access is the best choice. For example, if we have to modify a value in record no 21, then using random access techniques, we can place the file pointer at the beginning of record 21 and then straight-way process the record. If sequential

access is used, then we'll have to unnecessarily go through first twenty records in order to reach at record 21.

Sequential Input and Output Operations: The file stream classes support a number of member functions for performing the input and output operations on files. Various functions used for sequential operation of files include **get(), put(), getline(), read() and write().** The functions get() and put() are capable of handling a single character at a time. These functions are byte-oriented. That is, get() will read a byte of data and put() will write a byte of data. The function getline() is used to handle multiple characters at a time. Another pair of functions i.e., read() and write() are capable of reading and writing blocks of binary data. The prototypes of these functions are as follows:

```
istream & get(char & ch) ;                                  // #1
istream & get(char * buf, int num, char delim = '\n') ;     // #2
int get() ;                                                 // #3
istream & getline(char * buf, int num, char delim = '\n') ;
istream & read ( (char *) & buf, int sizeof(buf) ) ;
ostream & write ( (char *) & buf, int sizeof(buf) ) ;
ostream & put(char ch) ;
```

The first form **(#1) of get()** reads a single character from the associated stream and puts that value in ch. It returns a reference to the stream. The put() writes the value of ch to the stream and returns a reference to the stream.

The second form **(#2) of get()** reads characters into a character array pointed to bye buf until either num characters have been read, or the character specified by delim has been encountered. For example, the following statements :

```
char line[40] ;
cin.get(line, 40, '$') ;
```

will read characters into line until either 40 characters are read or '$' character is encountered, whichever occurs earlier. That is, if the input given in response to above statement is as follows :
Value is $ 177.5 then line will be storing **value is**
And if the input given is **The amount is 17.5** The contents of line will be **The amount is 17.5**

The array pointed to by buf (any user defined name) will be null-terminated by get(). If no delim character is specified, by default a newline character acts as a delimiter. If the delimiter character is encountered in the input stream, the get() function does not extract it. Rather, the delimiter character remains in the stream until the next input operation.

The third form **(#3) of get()** returns the next character form the stream. It returns EOF if the end of the file is encountered.

This **getline()** is virtually identical to get(buf, num, delim) version of get(). The function getline() also reads characters from input stream and puts them in the array pointed to by buf until either num character have been read, or the character

specified by delim is encountered. If not mentioned, the default value of delim is newline character.

The **read() and write()** Functions are used for reading and writing blocks of binary data. The read() function reads sizeof(buf) (it can be any other integer value also) bytes from the associated stream and puts them in the buffer pointed to by buf. The write() function writes sizeof(buf) (it can be any other integer value also) bytes to the associated stream from the buffer pointed to by buf. These functions take two arguments. The first is the address of variable buf, and the second is the length of that variable in bytes. The address of the variable must be type cast to type char * (i.e., a pointer to character type). The data written to a file using write() can only be read accurately using read().

The following program writes a structure to the disk and then reads it back by using write() and read() functions :

/* C++ Sequential Input and Output Operations with Files */

```
#include<iostream>
#include<fstream>
#include<string>
#include<stdlib>
#include<conio>
using namespace std;
struct customer {
        char name[20];
        float balance;
};
void main() {
  clrscr();
  customer savac;
  cout<<"Enter your name: ";
  cin.get(savac.name, 20);
  cout<<"Enter balance: ";
  cin>>savac.balance;
ofstream fout;
fout.open("Saving", ios::out |
        ios::binary);
if(!fout)  {
        cout<<"File can\'t be opened..!!\n";
        cout<<"Press any key to exit...\n";
        getch(); exit(1);
}
fout.write((char *) & savac, sizeof(customer));    // write to file
fout.close();          // close connection
// read it back now
ifstream fin;
fin.open("Saving", ios::in | ios::binary);    // open input file
fin.read((char *) & savac, sizeof(customer));     // read structure
cout<<savac.name<<" has the balance amount of Rs.
"<<savac.balance<<"\n";
fin.close();
cout<<"\nPress a key to exit...\n";  getch();
}
Output:
Enter your name: Alok
Enter balance: 65000
Alok has the balance amount of Rs. 65000
Press any key to exit...
```

Here, Only a single call to read() and write() is necessary to read or write the entire structure. Each individual field need not be read or written separately. If the end of

the file is reached before the specified number of characters have been read, the read() simply stops, and the buffer contains as many characters as were available.

Reading and Writing Class Objects: The functions **read() and write()** can also be used for reading and writing class objects. These functions handle the entire structure of an object as a single unit, using the computer's internal representation of data. For example, the function write() copies a class object from memory byte by byte with no conversion. But one thing that must be remembered is that only data members are written to the disk file and not the member functions. The length of an object is obtained by sizeof operator and it represents the sum total of lengths of all data members of the object. Here is an example:

// Example: Sequential Input and Output Operations with Files

```
#include<iostream.h>
#include<fstream.h>
#include<conio.h>
#include<stdlib.h>
class student {
   char name[20], grade;
   float marks;
public:
   void getdata(void);
   void display(void);
};
void student::getdata(void) {
   char ch;
   cin.get(ch);
   cout<<"Enter name: ";
   cin.getline(name, 20);
   cout<<"Enter grade: ";
   cin>>grade;
   cout<<"Enter marks: ";
   cin>>marks;
}
void student::display(void) {
```

```
   fio.open(fname, ios::in || ios::out);
   if(!fio)  {
            cout<<"Error occurred in opening the file..!!\n";
            cout<<"Press any key to exit...\n";
            getch();   exit(1);
   }
   cout<<"Enter details for 3 students:\n\n";
   for(int i=0; i<3; i++) {
            cse[i].getdata();
            fio.write((char *)&cse[i], sizeof(cse[i]));
   }
   fio.seekg(0);     /* seekg(0) resets the file to start, to access the
file * from the beginning */
   cout<<"The content of "<<fname<<" file are shown below:\n";
   for(i=0; i<3; i++) {
            fio.read((char *)&cse[i], sizeof(cse[i]));
            cse[i].display();
   }
   fio.close();
   cout<<"\nPress any key to exit...\n";  getch();
}
```

Output:

```
   cout<<"\nName: "<<name;
   cout<<"\tGrade: "<<grade;
   cout<<"\tMarks: "<<marks;
}
void main() {
   clrscr();
   char fname[20];
   student cse[3];
   fstream fio;
   cout<<"Enter file name: ";
   cin.get(fname, 20);
```

```
Enter file name: myfile.txt
Enter details for 3 students:

Enter name: Alok
Enter grade: A
Enter marks: 85

Enter name: Deepak
Enter grade: B
Enter marks: 72

Enter name: Rupam
Enter grade: C
Enter marks: 65

The content of myfile.txt file are shown below:
name: Alok          grade: A      marks: 85
name: Deepak        grade: B      marks: 72
name: Rupam         grade: C      marks: 65

Press any key to exit...
```

Updating a File: Random Access: Updating of file include one or more of the following tasks –

- Displaying the contents of a file
- Modifying an existing item
- Adding a new item
- Deleting an existing item

In C++, random access is achieved by manipulating seekg(), seekp(), tellg() and tellp() functions. The seekg() and tellg() functions are used to set and examine the get_pointer, and the seekp() and tellp() functions perform these operations on the put_pointer. The seekg() and tellg() functions are for input streams (ifstream) and seekp() and tellp() functions are for output streams (ofstream). However, if we use them with an fstream object then tellg() and tellp() return the same value. Also seekg() and seekp() work the same way in an fstream object. The most common forms of these functions are shown here:

seekg()	istream & seekg(long); istream & seekg(long, seek_dir);	Form 1 Form 2
seekp()	ofstream & seekp(long); ofstream & seekp(long, seek_dir);	Form 1 Form 2
tellg()	long tellg()	
tellp()	long tellp()	

The working of seekg() & seekp() and tellg() & tellp() is just the same except that seekg() and tellg() work for ifstream objects and seekp() and tellp() work for ofstream objects. In the above table, seek_dir takes the definition enum seek_dir { beg, cur, end};. The seekg() or seekp(), when used according to Form 1, then it moves the get_pointer or put_pointer to an absolute position.

Here is an example:

```
ifstream fin;
ofstream fout;
:                       // file opening routine
fin.seekg(30);          // will move the get_pointer (in ifstream) to byte number 30 in the file
fout.seekp(30);         // will move the put_pointer (in ofstream) to byte number 30 in the file
```

When seekg() or seekp() function is used according to Form 2, then it moves the get_pointer or put_pointer to a position relative to the current position, following the definition of seek_dir. Since, seek_dir is an enumeration defined in the header file iostream.h, that has the following values:

```
ios::beg        // refers to the beginning of the file
ios::cur        // refers to the current position in the file
ios::end        // refers to the end of the file
```

For example -

```
fin.seekg(30, ios::beg);        // go to byte no. 30 from beginning of file linked with fin
fin.seekg(-2, ios::cur); // back up 2 bytes from the current position of get pointer
fin.seekg(0, ios::end); // go to the end of the file
fin.seekg(-4, ios::end);        // backup 4 bytes from the end of the file
```

The functions tellg() and tellp() return the position, in terms of byte number, of getpointer and put_pointer respectively, in an input file and output file.

//File Pointers and Random Access Example

```
#include<fstream>
#include<conio>
#include<stdlib>
#include<stdio>
#include<string>
using namespace std;
class student {
  int rollno;
  char name[20], branch[3],
  float marks;
  char grade;
public:
void getdata() {
  cout<<"Rollno: ";       cin>>rollno;
  cout<<"Name: ";         cin>>name;
  cout<<"Branch: ";       cin>>branch;
  cout<<"Marks: ";        cin>>marks;
  if(marks>=75)           grade = 'A';
  else if(marks>=60)      grade = 'B';
  else if(marks>=50)      grade = 'C
  else if(marks>=40)      grade = 'D';
  else                    grade = 'F';
}
void main() {
  clrscr();
  fstream fio("marks.dat", ios::in |
  ios::out);
  char ans='y';
  while(ans=='y' || ans=='Y') {
    stud1.getdata();
     fio.write((char *)&stud1,
     sizeof(stud1));
    cout<<"Record added to the file\n";
    cout<<"\nWant to enter more ?
     (y/n)..";
    cin>>ans;
  }
  clrscr();
  int rno;
  long pos;
  char found='f';
  cout<<"Enter rollno of student whose
  record is to be modified: ";
  cin>>rno;
  fio.seekg(0);
  while(!fio.eof()) {
```

```
 void putdata() {
   cout<<"\nRollno:"<<rollno<<"\tName: "<<name;
   cout<<"\nMarks: "<<marks<<"\tGrade: "<<grade";
 }
 int getrno() {   return rollno;   }
 void modify();
 }stud1, stud;

 void student::modify()  {
   cout<<"\nRollno: "<<rollno<<"\tName: "<<name ";
   cout<<"\nBranch: "<<branch<<"\tMarks: "<<marks;
   cout<<"Enter new details.\n";
   char nam[20]=" ", br[3]=" ";
  float mks;
   cout<<"New name:(Enter '.' to retain old one): ";
   cin>>nam;
   cout<<"New branch:(Press '.' to retain old one): ";
   cin>>br;
   cout<<"New marks:(Press -1 to retain old one): ";
   cin>>mks;

   if(strcmp(nam, ".")!=0)   strcpy(name, nam);
   if(strcmp(br, ".")!=0)    strcpy(branch, br);
   if(mks != -1)             marks = mks;
   if(marks>=75)             grade = 'A';
   else if(marks>=60)        grade = 'B';
   else if(marks>=50)        grade = 'C';
   else if(marks>=40)        grade = 'D';
   else                      grade = 'F';
 }
```

```
    pos = fio.tellg();
    fio.read((char *)&stud1,
     sizeof(stud1));
    if(stud1.getrno() == rno)   {
          stud1.modify();
          fio.seekg(pos);
          fio.write((char *)&stud1,
          sizeof(stud1));
          found = 't';
          break;
    }
  }
  if(found=='f')    {
          cout<<"\nRecord not found in the
          file..!!\n";
          cout<<"Press any key to exit...\n";
          getch();
          exit(2);
  }
  fio.seekg(0);
  cout<<"Now the file contains:\n";
  while(!fio.eof()) {
          fio.read((char *)&stud,
          sizeof(stud));
          stud.putdata();
  }
  fio.close();
  getch();
}
```

Error Handling during File Operations: Sometimes during file operations, errors may creep in because of the following reasons.

- A file being opened for reading might not exist.
- A file name used for a new file may already exist.
- An attempt could be made to read past the end-of-file.
- Invalid operation performed on a file which has not been opened in appropriate mode.
- There might not be enough space in the disk for storing data.

To check for such errors and to ensure smooth processing, C++ file streams inherit 'stream-state' members from the ios class that store the information on the status of a file that is being currently used. The current state of the I/O system is held in an integer, in which the following flags are encoded :

Name of the flag	Meaning
eofbit	1 when end-of-file is encountered, 0 otherwise.
failbit	1 when a non-fatal I/O error has occurred, 0 otherwise
badbit	1 when a fatal I/O error has occurred, 0 otherwise

goodbit	0 value

There are several error handling functions supported by class **ios** that help to read and process the status recorded in a file stream. Following table lists these error handling functions and their meaning :

Function	Meaning
int bad()	Returns a non-zero value if an invalid operation is attempted or any unrecoverable error has occurred. However, if it is zero (false value), it may be possible to recover from any other error reported and continue operations.
int eof()	Returns non-zero (true value) if end-of-file is encountered while reading; otherwise returns zero (false value).
int fail()	Returns non-zero (true) when an input or output operation has failed.
int good()	Returns non-zero (true) if no error has occurred. This means, all the above functions are false. For example, if fin.good() is true, everything is okay with the stream named as fin and we can proceed to perform I/O operations. When it returns zero, no further operations can be carried out.
clear()	Resets the error state so that further operations can be attempted.

Example:

```
ifstream fin;
fin.open("master", ios::in);
if(!fin.fail()) {          :      // process the file        }
if(fin.eof()) {            :       // terminate the program        }
else if(fin.bad()) {       :       // report fatal error       }
     else {
               fin.clear();    // clear error-state flags
               :
         }
:
```

Command-line Arguments: If any input value is passed through command prompt at the time of running of program, it is known as **command line argument**. It is a concept of passing the arguments to the main() function by using command prompt.

To pass command line arguments, we define the main() with two arguments; first argument is the number of command line arguments and second is list of command-line arguments.

int main(int argc, char *argv[ ]) { /* ... */ } or
int main(int argc, char **argv) { /* ... */ }

- **argc (ARGument Counter)** is of type **int** and stores number of command-line arguments passed by the user including the name of the program. So if we pass a value to a program, value of argc would be 2 (one for argument and one for program name). The value of argc should be non negative.
- **argv(ARGument Vector)** is array of character pointers listing all the arguments. If argc is greater than zero, the array elements from argv[0] to argv[argc-1] will contain pointers to strings. Argv[0] is the name of the program.

Example - // Name of program is mainreturn.cpp

```
#include<iostream>
using namespace std;
int main(int argc, char* argv[ ]) {
   cout << "You have entered " << argc << " arguments: ";
   for (int i = 0; i < argc; i++)
      cout << argv[i] << "\n";
   return 0;
}
```

Output:

```
mainreturn hello world
You have entered 3 arguments: mainreturn hello world
```

Note: We pass all the command line arguments separated by a space, but if argument itself has a space then we can pass such arguments by putting them inside double quotes or single quotes.